

HoBBY: Hardening Unbalanced Branches against Control Flow Attacks on Intel SGX and AMD SEV

Chang Liu^{†*}, Shuaihu Feng^{†*}, Yuan Li^{‡¶}, Dongsheng Wang^{†‡}, Trevor E. Carlson[§]

[†]Tsinghua University, Beijing, China

[‡]Zhongguancun Laboratory, Beijing, China

[§]National University of Singapore, Singapore

Abstract—This paper introduces HoBBY, a compiler-based tool that hardens unbalanced branches at the instruction level, making parallel control flows indistinguishable to state-of-the-art attacks that bypass the source-code level balancing. To achieve this, we propose a single-step analysis method to identify unbalanced instructions in secret-dependent branches, and implement instruction shadowing, cogging, and spiraling techniques to protect them. We evaluate HoBBY by hardening secret-dependent branches in four real-world applications, validating its resilience against three state-of-the-art attacks targeting Intel SGX and AMD SEV. HoBBY achieves a runtime overhead of 2.8% for cryptographic libraries and a binary size overhead of 0.6%.

I. INTRODUCTION

Modern CPUs incorporate Trusted Execution Environments (TEEs), such as Intel Software Guard Extensions (SGX) [16] and AMD Secure Encrypted Virtualization (SEV) [3], to safeguard the instructions and data of applications, preventing data leakage from malicious programs, including the operating system. However, TEE designers do not comprehensively address side-channel threats [27], which has led to various control flow attacks [5], [9], [26] that expose control flow information, such as the target of a secret-dependent branch, posing significant risks to applications like cryptographic libraries used for RSA [23], [39] and ECDSA [22] algorithms.

To defend against these attacks, researchers propose constant-time programming, which maintains constant execution time regardless of the specific data being processed. Two primary methods are available for implementing constant-time programming. The first approach is to avoid making control flow directly dependent on secrets [7], [15], thereby preventing control-flow attacks by eliminating associations between sensitive data and control flow. However, this technique is often limited to specific cryptographic algorithms [9] and may introduce significant performance overhead [7], [26]. The second method is branch balancing, also known as code obfuscation [1], [36], which ensures equal execution time across branch paths, making it difficult for attackers to distinguish between paths using CPU side channels. Branch balancing is algorithm-agnostic and preserves program semantics, enhancing its deployability.

However, existing branch balancing implementations still expose vulnerabilities and have been bypassed by recent attacks [5], [25], [26], primarily due to the following three

reasons. First, current branch balancing techniques are designed mainly to counter traditional power, frequency and timing side-channel attacks [35], [41], ensuring only that the overall execution time of two branch paths remains consistent. They overlook that attackers with root privileges have access to advanced techniques, such as single-step execution [30], [31], [37], which enable detection of information at the instruction level. Second, current branch balancing is typically implemented at the source-code level [22], [23] and does not account for imbalances introduced during compilation, which may introduce new attack surfaces [25], [32]. Third, existing branch balancing does not fully consider the influence of the Program Counter (PC) [36]. For example, even if two branches contain identical instructions, differing PCs can still impact CPU behavior in ways exploitable by attackers [5], [26]. Currently, no approach has been proposed to harden secret-dependent branches at the instruction level.

In this paper, we propose HoBBY, an algorithm-agnostic, compile-based software defense that Hardens unBalanced BranBes at the instruction level. By ensuring consistency in instructions (i.e., undistinguishable instruction types, counts, data flow, and PC targets) on both branch paths during compilation, HoBBY eliminates side-channel information exploitable by attackers, effectively defending against such threats. To achieve this, HoBBY first performs single-step analysis through instrumentation to evaluate the balance of secret-dependent branches within applications, identifying unbalanced instruction sequences that may be vulnerable to side-channel leakage. Next, HoBBY employs instruction and data shadowing, replicating instructions and registers without altering program semantics, to maintain balance in instruction type, count, and data flow across both branch paths. Finally, HoBBY adjusts the layout of original and shadow instructions within memory pages, merging store and load instructions to maintain the PC targets balance.

We implement HoBBY with LLVM [21] and apply it to harden the unbalanced branches in three cryptographic [22], [23], [39] and one non-cryptographic applications [19]. To assess HoBBY's attack resilience, we show that HoBBY can drop the accuracy of three state-of-the-art attacks, two on Intel SGX [5], [28] and one on AMD SEV [9], to the equivalent of random guessing (i.e., approximately 50%). Finally, we evaluate HoBBY's performance overhead, showing that HoBBY introduces an average runtime overhead of 2.8% for

* These authors contributed equally to this work.

¶ Corresponding author.

the cryptographic libraries, with a binary size overhead of 0.6%, which is much lower compared to other mitigations [7].

In summary, the contributions of this paper are as follows:

- We introduce HoBBY, a tool that hardens unbalanced branches at the instruction level in real-world applications, and defends against state-of-the-art control flow attacks on these applications.
- We propose a method to evaluate the security of secret-dependent branches, enabling automated analysis of unbalanced instructions along branch paths and identifying potential side-channel risks.
- We develop various methods to balancing branch in terms of instruction type, count, data address and PC target.
- We evaluate HoBBY's effectiveness and performance overhead in 4 real-world applications, demonstrating that HoBBY defends against state-of-the-art attacks on Intel SGX and AMD SEV with an average runtime overhead of approximately 2.8% for the cryptographic libraries, and a binary size overhead of approximately 0.6%.

II. BACKGROUND ON CONTROL FLOW ATTACKS

Control flow side-channel attacks infer which code of an application running in the TEE is executed by detecting changes in architecture-level or microarchitecture-level states, thereby leaking secrets related to control flow. Secret-dependent branches [5], [14] are one of the primary targets of such attacks, as the execution of these branches depends on a secret determining whether the branch is taken or not. Secret-dependent branches are commonly found in cryptographic applications, such as RSA and ECDSA [22], [23], [39], and are also present in non-cryptographic applications [18], [19].

Current control flow attacks targeting secret-dependent branches can be divided into two categories. The first category is branch-oriented attacks [14], [17], [42], [43], which exploit the effects of branch instructions on the branch predictor to leak sensitive data. The second category is non-branch-oriented attacks [5], [9], [25], [26], [28], [40], which infer the branch direction by exploiting side channels of other instructions along the branch paths.

For branch-oriented attacks, the leakage source is known, and defenses can be implemented by flushing the branch predictor during context switches [42], [43] or confounding the control flow through compiler [12], [17]. Therefore, such attacks are not considered in this paper. For non-branch-oriented attacks, imbalances between the two branch paths, such as differences in instruction count [25], instruction types [2], [9], data addresses [12], [28], or instruction addresses (PC) [5], [26], are exploited to infer branch paths, as shown in Table I. This type of attack leverages various complex microarchitectural side channels and diverse code patterns, posing significant challenges to effective defenses.

III. OVERVIEW OF HOBBY

A. Motivation

Branch balancing prevents various side-channel attacks by ensuring identical CPU behavior on both sides of a branch.

TABLE I
NON-BRANCH-ORIENTED CONTROL FLOW ATTACKS ON
SECRET-DEPENDENT BRANCHES IN INTEL SGX AND AMD SEV. PAGE
MEANS PAGE ATTACKS OR CONTROLLED CHANNEL ATTACKS, WHILE
CACHE REFERS TO CACHE SIDE-CHANNEL ATTACKS.

Attack Type / Name	Source	Imbalance Requirement
Page [12], [33], [34], [40]	Interrupt	Data Flow & PC (4 KiB)
Nemesis [32]	Interrupt	Type
Copycat [25]	Interrupt	Count
Frontal Attack [26]	Frontend	PC (Store)
TLBleed [11]	TLB	Data Flow & PC (4 KiB)
PortSmash [2]	Port	Type & Count
Cache [4], [13], [24], [28]	Cache	Data Flow & PC (64 B)
AfterImage [5]	Prefetcher	PC (Load)
SQUIP [9]	Scheduler	Type & Count

This approach does not modify program algorithms and has therefore already been adopted by some developers [22], [23], [39]. However, current branch balancing is primarily implemented at the source-code level, without addressing imbalances introduced during compilation [25] or leakage from instruction PCs due to specific CPU characteristics [5], [26]. As a result, branch balancing is not always effective against control flow attacks. In this work, we look at the fundamental root causes of branch balancing issues and propose new compiler-based techniques to eliminate branch-balancing issues. Specifically, we aim to strengthen balanced branches at the instruction level to defend against state-of-the-art non-branch-oriented attacks, such as those listed in Table I.

B. Assumptions and Threat Model

We assume an attacker with root privileges (e.g., a malicious OS) aims to leak the direction of a secret-dependent branch within the TEE application using control flow attacks. The attacker can manage page tables and handle exceptions to execute victim code in single-step mode (e.g., using SGX-Step [31] or SEV-Step [37]) and perform side-channel attacks. The victim is a user application running in the TEE, and seeks to harden its code against control flow attacks through compiler-based defenses. Both the victim and attacker can profile the binary file and identify secret-dependent branches. The victim can compile, modify, and execute the binary, while the attacker can only analyze the binary.

C. Workflow of HoBBY

The workflow of HoBBY is illustrated in Fig. 1. The input is the source code containing secret-dependent branches that require hardening. First, HoBBY automatically identifies these branches and their paths. Next, HoBBY instruments each instruction along the branch paths to analyze the context of each instruction. After the analysis, HoBBY balances the type, count, data flow, and PC of instructions along the branch paths to defend against control flow attacks. The output is a hardened application, preventing privileged attackers from using side channels to distinguish between different branch paths.

In Sections IV and V, we introduce the design of secret-dependent branch analysis (❶) and branch balancing (❷),

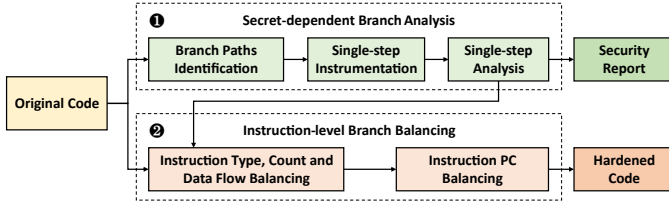


Fig. 1. Workflow of HoBBY. HoBBY first identify and evaluate the security of secret-dependent branches (①), and then balancing the instruction type, count, data flow and PC target (②).

respectively. In Section VI, we present the effectiveness and performance evaluation of HoBBY.

IV. SECRET-DEPENDENT BRANCHES ANALYSIS

In this section, we introduce HoBBY’s single-step analysis of secret-dependent branches. This step identifies unbalanced and potentially vulnerable instruction sequences, which will be hardened in subsequent steps.

A. Branch Paths Identification

Similar to previous research [8], we use DFSan [20] to perform taint analysis for identifying secret-dependent branches. Next, we analyze the program’s Control Flow Graph (CFG) to determine the control flow in both directions of each branch. For most secret-dependent branches, only a small portion of the secret (e.g., a single bit) is processed per execution, causing these branches to be executed multiple times. Consequently, there is often a common successor across different paths for the majority of secret-dependent branches. Based on this observation, we first use Breadth-First Search (BFS) to identify the nearest common successor among different branch paths.

Subsequently, we use Depth-First Search (DFS) to trace all paths from the basic block containing the secret-dependent branch to the nearest common successor. During DFS, other branches may be encountered along these paths (e.g., branches for loops or exception handlers [23], [39]), adding complexity to the control flow. To handle loops, we perform a static analysis to partition the loop into the loop head and loop body, ensuring that each loop body is analyzed only once. For branches handling exceptions, we apply a heuristic approach: if the number of predecessors for the branch target exceeds a specified threshold, and the basic block’s successor is unreachable from the nearest common successor on the branch path, we skip the analysis of that basic block and its successors.

B. Single-step Instrumentation

According to the threat model described in Section III-B, the attacker can interrupt the victim’s code [45] at the instruction level [30] and perform side-channel attacks. To analyze the vulnerability of branches, we conduct a dynamic single-step analysis on the instructions along the branch paths after identifying them. To achieve this, we insert a `call` instruction targeting the function `step` between each instruction, as shown in Fig. 2(a). In each `step`, we save the current context by sequentially storing the values of the `EFLAGS` register and general-purpose registers, followed by values in the call stack.

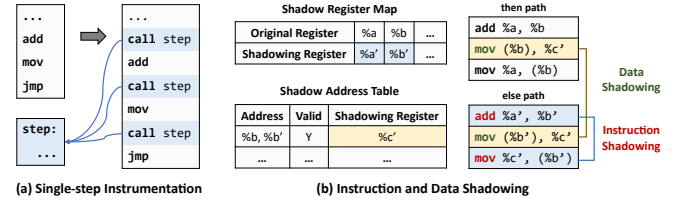


Fig. 2. Methods to perform single-step analysis at the instruction level (a), and to implement instruction and data shadowing (b) so that instruction types and memory accesses are undistinguishable on the two branch paths.

Additionally, for store and load instructions, we capture their virtual addresses for later offline analysis.

To ensure that `step` does not alter the program’s context, we save the current stack frame register `RSP` before executing `step`, then adjust it to a new stack space to store the execution context. Furthermore, to retrieve information about the executing instruction, we additionally save the PC stored in `RIP`. Given that the length of the instrumented instructions is known, we can dynamically calculate the PC of each analyzed instruction and access the corresponding instruction page to retrieve its machine code. Finally, after `step` completes, we restore the context and adjust the stack base to ensure the program continues to execute in single-step mode.

C. Analysis on Unbalanced Instructions

After collecting the context of each instruction along the branch paths, we perform an offline analysis to determine whether the instructions are balanced and, if not, identify which instructions might be vulnerable to side-channel attacks. First, we disassemble the machine code to obtain the opcodes and then count the number of each opcode type on each path. If the instruction counts differ, or if there are different types of instructions, the branch may be vulnerable to port-based [2], [9] or interrupt-based attacks [25], [32]. Next, we analyze the presence of load and store instructions within the branch paths. If such instructions exist, the branch could be vulnerable to the attacks targeting the PC of these instructions [5], [26]. Finally, we analyze the operands of these instructions to determine whether the addresses for loads and stores are in different pages or cache lines, where hardening is required because distinct data flows may be vulnerable to side-channel attacks related to page [40], TLB [11], and cache [10], [28].

V. HARDENING UNBALANCED BRANCHES

In this section, we introduce HoBBY’s methods for hardening unbalanced instruction types, instruction counts, data flow, and the PC targets. Even if an attacker can execute the victim code in single-step mode, identical instructions from the victim program are executed, and the same data addresses are accessed, which effectively prevents these control flow attacks.

A. Instruction Balancing with Instruction Shadowing

HoBBY first balances instruction types and counts through *instruction shadowing*. As shown in Fig. 2(b), we start by creating a shadow register mapping table that maps all register operands in unbalanced instructions to shadow registers.

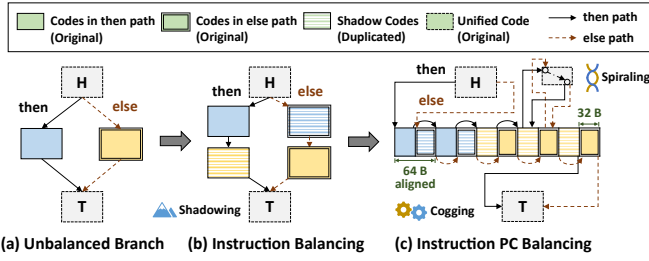


Fig. 3. Workflow of hardening the unbalanced branches. First, the instruction type, count and data flow are balanced through instruction and data shadowing (b). Next, the instruction PC target is balanced through instruction cogging and instruction spiraling (c).

Shadow registers are registers outside the program’s original semantics and are not used as operands in the original instructions, and thus do not affect the program’s behavior.

As shown in Fig. 3(b), if there are imbalances in instruction types or counts between the two branch paths, we replicate the basic blocks. We then insert new shadow instructions into the replicated basic blocks, adjusting the instruction opcodes to ensure that both branch paths contain the same number and types of specific instructions. When adding these instructions, we maintain the same relative position from the entry of each basic block so that, during the single-step execution, identical instruction types are executed on both branch paths. For shadow instructions, we ensure their target registers are always shadow registers, preventing the execution of these instructions from impacting the program’s original context. For example, as shown in Fig. 2(b), if an `add` instruction exists on one branch path but not on the other, we replicate the basic block on the other path and add a shadow `add` instruction with operands drawn from the shadow register map.

B. Data Flow Balancing with Data Shadowing

Since store and load instructions trigger data accesses and impact CPU backend units such as the cache, we need to balance their memory addresses to avoid data flow imbalance through *data shadowing*. For load instructions, we access the same address in the shadow instructions. For store instructions, to prevent shadow instructions from modifying memory, we insert a shadow load before the shadow store to retrieve the original data from that address, as shown in Fig. 2(b). To balance the data flow, we also insert a load on the original path that accesses the same address but uses a shadow register as the destination, ensuring program correctness is maintained.

C. PC Target Balancing with Instruction Cogging

In modern CPUs, instructions are fetched from memory, so even if the types and counts of instructions are identical on both branch paths, instruction fetching can still affect the cache and TLB, potentially creating side-channel leakage risks [33]. Therefore, it is necessary to balance the instruction PC target to ensure that both original and shadow instructions reside on the same page and access the same instruction cache line.

To achieve this, we introduce the *instruction cogging* technique. As shown in Fig. 3(c), after completing instruction

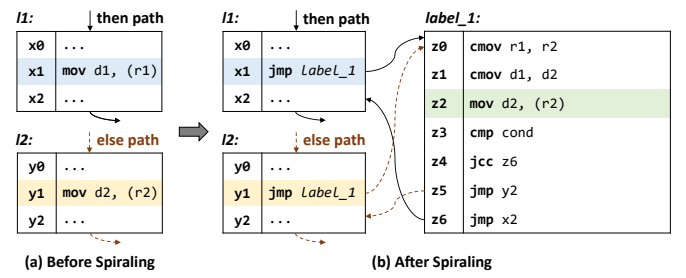


Fig. 4. Method to implement instruction spiraling. The load or store instructions on two branch paths can be merged to hide the side-channel effects caused by these instructions with different PCs.

balancing, we divide the instructions on both branch paths into 32-byte blocks and alternately place these blocks within a 64-byte aligned address space, connecting the two paths with `jmp` instructions to maintain semantic correctness. This ensures that, even if an attacker executes instructions in single-step mode, balanced instructions from both paths appear simultaneously within the same instruction page and cache line, preventing related side-channel attacks.

D. PC Target Balancing with Instruction Spiraling

Recent attacks exploit the different effects of stores or loads with different PCs on CPU microarchitecture to infer which stores or loads are executed on different paths. For example, the Frontal Attack [26] leverages variations in single-step execution time for a store based on its distance from the conditional branch. AfterImage [5] takes advantage of loads with different 8 least significant bits (LSBs) in their PCs, which update distinct entries in the IP-stride prefetcher.

To defend against such attacks, only balancing the instruction and data flow is insufficient. Therefore, we propose the *instruction spiraling* technique, which merges the store and load instructions from both paths, as shown in Fig. 4. In the instruction balancing phase, we have ensured balanced store and load instructions and data flow on both branch paths using instruction and data shadowing. Now for each pair of balanced stores or loads, we create a new basic block in the CFG and replace the original stores or loads on both paths with `jmp` instructions targeting this new basic block.

In this new basic block, we insert a `cmov` instruction that, based on the branch condition, determines the source and destination registers for the stores or loads. This setup ensures that the same store or load with a fixed PC is executed on both paths each time, producing consistent effects on CPU microarchitecture. To maintain correct control flow, the program jumps back to one of the original paths based on the branch condition after executing the store or load in the new basic block, ensuring that execution proceeds as intended.

VI. EVALUATION ON HOBBY

We implement HoBBY in the LLVM backend [21] for the x86-64 instruction set. Specifically, we include single-step analysis, instruction shadowing, data shadowing, and instruction spiraling for stores and loads. Our implementation does not cover 32-bit instructions, extended instruction sets, or

TABLE II
SINGLE-STEP ANALYSIS ON THE UNBALANCED INSTRUCTIONS OF 8 SECRET-DEPENDENT BRANCHES IN 4 REAL-WORLD APPLICATIONS BEFORE HARDENING AND AFTER HARDENING. THE LAST 2 COLUMNS SHOW THE AVERAGE RUNTIME OVERHEAD OF THE HARDENING FOR THE FUNCTIONS THAT CONTAIN THE SECRET-DEPENDENT BRANCHES, AND THE WHOLE APPLICATIONS.

Application	Secret-dependent Branch Location	Imbalance Before Hardening				Imbalance After Hardening	Performance Overhead	
		Count	Type	Data Flow	PC		Function	Program
MbedTLS v2.6.1 [22]	bignum.c, line 1013	1	2	0	14	0	14.3%	2.8%
	bignum.c, line 1044	1	2	0	14			
	bignum.c, line 1581	0	0	0	10			
MbedTLS v3.6.1 [23]	bignum.c, line 1823	1	0	0	10	0	5.2%	0.5%
	bignum.c, line 1959	0	0	0	18			
WolfSSL v5.7.2 [39]	integer.c, line 1329	0	0	0	18	0	0.4%	0.2%
Libjpeg v9f [19]	jdctint.c, line 208	69	7	42	27	0	66.4%	19.3%
	jdctint.c, line 325	71	5	43	33			

system instructions, as these are not present in the programs we analyzed. We leave these aspects for future work. In this section, we selected Libjpeg [19], WolfSSL [39], and MbedTLS [22], [23] for analysis, because recent research has identified secret-dependent branches in these applications as attack targets. We use single-step analysis before and after hardening to demonstrate HoBBY's effectiveness. We then show that HoBBY can defend against three state-of-the-art attacks targeting Intel SGX and AMD SEV. Finally, we present the performance overhead of HoBBY.

A. Effectiveness

In Table II, we select a few secret-dependent branches from each program and report the number of unbalanced instructions before and after hardening. The developers of WolfSSL and MbedTLS have considered side-channel attacks and implemented source-code level balancing (i.e., apart from function parameters, the code on both sides of the branches is identical). However, these branches still exhibit risks due to unbalanced instructions [5], [25], [26]. Through instruction balancing and PC target balancing, HoBBY eliminates the imbalance and mitigates potential side-channel risks.

As an example from a non-cryptographic library, Libjpeg does not consider branch balancing during development. Instead, Libjpeg's IDCT algorithm uses branching to skip complex computations when the image data is simple (e.g., a pixel block consists of a single color), and HoBBY efficiently hardens these unbalanced instructions.

B. Case Studies

In this section, we select three state-of-the-art control-flow attacks to demonstrate the effectiveness of HoBBY. These attacks exploit imbalances in data flow, instruction count, and PC, respectively. For the victim, we use the secret-dependent branches in the modular inverse function of MbedTLS [23]. We do not implement a full end-to-end attack; instead, we achieve synchronization by actively switching between attacker and victim. This approach simplifies the attack process but is sufficient to demonstrate HoBBY's effectiveness.

Cache Attacks [4], [28] exploit the imbalance in the data flow. If the victim's load instructions on different branch paths access different data addresses, an attacker can use cache side channels to determine which load is executed. As shown in Fig. 5(a), if two loads access distinct cache sets, the attacker can distinguish which cache set the victim accessed after each branch execution using Prime+Probe [28], achieving a 99.75% success rate. HoBBY applies data shadowing to ensure that loads on both paths access the same cache sets, so the attacker always observes both cache sets as accessed, rendering the attack ineffective (i.e., equivalent to random guessing).

SQUIP [9] leverages the difference in the number of arithmetic instructions (e.g., multiplications) on different branch paths to create contention in the scheduler of AMD CPU execution units, thereby revealing which path the victim's code take. As shown in Fig. 5(b), the attacker's code exhibits different execution times for victim paths with varying numbers of multiplications, resulting in a 92.77% attack success rate. After hardening, the instruction counts on both paths are identical, and the execution time of the attacker's code becomes the same, reducing the attack success rate to around 50%.

AfterImage [5] exploits the differing effects of loads with distinct PCs on the prefetcher to infer which load is executed. For example, if a load on a particular path is executed, the prefetcher entry associated with that load's PC is updated; otherwise, the prefetcher entry remains unchanged. By repeatedly executing the victim code and using a cache side channel to detect the prefetcher update rate, the attacker can infer whether the load is executed. As shown in Fig. 5(c), when the update rate is zero, the load is not executed, indicating that the branch does not follow that path, with a success rate of 98.89%. HoBBY effectively mitigates AfterImage by merging loads on both paths through instruction spiraling, so the attacker always observes a prefetcher update, making the attack ineffective.

C. Performance Overhead

We analyze the performance overhead of HoBBY on both the functions containing the secret-dependent branches and the

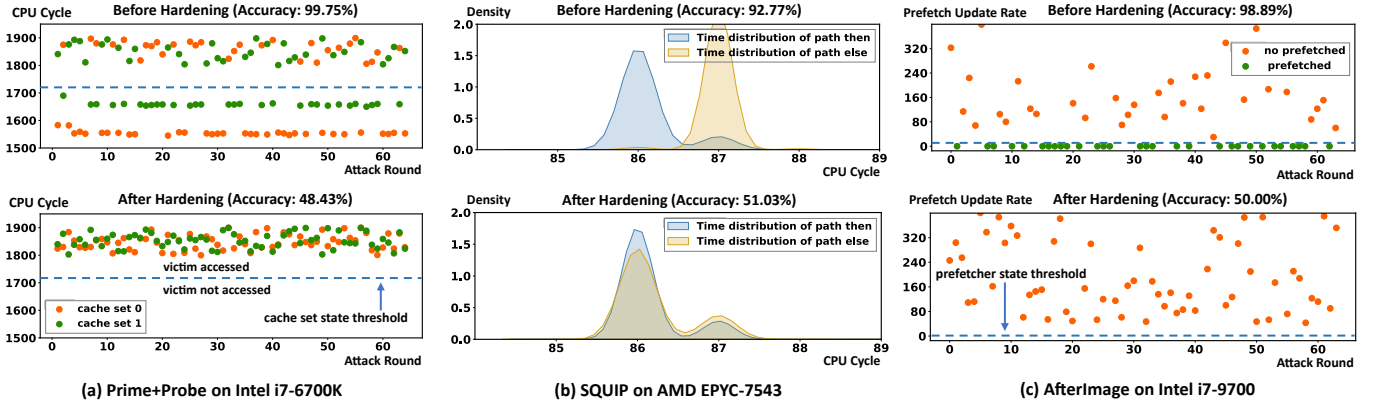


Fig. 5. Case studies on three state-of-the-art control flow attacks on Intel SGX and AMD SEV. HoBBY effectively defend these attacks by balancing the data flow [28], count of `imul` instructions [9] and load PC [5], making different paths of the secret-dependent branches not distinguishable.

overall program, as shown in the last two columns of Table II. For cryptographic libraries, we focus on the RSA algorithms as they are the most common attack targets in recent studies [5], [9], [28]. For Libjpeg, we focus on the IDCT function.

While hardening introduces runtime overhead, the overall performance impact remains acceptable given the security benefits. For instance, Libjpeg’s IDCT function sees a significant 66.4% overhead due to disrupted optimizations, but the total overhead for Libjpeg is only 19.3%, which is acceptable in many cases. In cryptographic libraries, if the hardened function isn’t on the critical execution path, the performance impact is minimal. For example, hardening the modular inverse function in MbedTLS results in just a 2.8% overall runtime overhead, indicating a negligible performance penalty.

We also evaluate the binary size overhead introduced by HoBBY. The additional instructions primarily stem from shadow instructions introduced by instruction and data shadowing and `jmp` instructions added by instruction spiraling. For branches that are not balanced at the source-code level (e.g., Libjpeg’s IDCT), the number of added instructions is significantly higher than in other programs. However, in terms of overall binary size, these additional instructions do not have notable effects. For example, the size increase in WolfSSL and Libjpeg due to HoBBY does not exceed 0.6%.

VII. RELATED WORK

Currently, various defenses have been proposed to mitigate control-flow attacks on Intel SGX and AMD SEV. First, as suggested by CPU vendors [15], preventing control flow from depending on secrets can fundamentally block all control-flow attacks [7]. However, this approach is difficult to extend to complex programs [9] and incurs significant overhead [7]. Second, code-oblivious techniques hide timing differences across different control flows. SGX-Shield [29] and Klotzski [44] achieve branch control-flow balancing at the page level but cannot defend against finer-grained control-flow attacks. Compiler-Assisted Hardening [38] and Obfuscuro [1] offer finer-grained instruction balancing but do not address leaks due to PC imbalances. Obelix [36] is a state-of-the-art tool that protects both code and data within the TEE, defending

against single-step control-flow attacks and ciphertext side channels [8] while also addressing PC imbalances [26]. However, Obelix primarily focuses on ensuring constant execution time for instructions during single-step execution and does not consider the persistent impact of specific instruction PCs on other CPU components, as highlighted in AfterImage [5].

In addition, some defenses are designed to mitigate specific attacks. First, by flushing the state of leakage sources during context switches, attacks with known sources can be mitigated, such as the BPU [42], [43] and Prefetcher [5] attacks. This approach is efficient and does not require modifications to the program, but is limited to specific attacks. Second, AEX-Notify [6] detects single-step attacks by monitoring the frequency of interrupts. However, it requires additional CPU support (e.g., microcode updates), which makes it unavailable for immediate deployment [36]. Besides, other TEEs (e.g., AMD SEV), currently lack support for similar techniques.

VIII. CONCLUSION

We present HoBBY, a tool that hardens unbalanced secret-dependent branches at the instruction level in applications running within a TEE. HoBBY balances instruction types, counts, data flow, and PC targets by employing single-step analysis, effectively defending against control-flow attacks on Intel SGX and AMD SEV. We demonstrate its effectiveness across four real-world applications, showcasing its ability to counter state-of-the-art control-flow attacks targeting these platforms. HoBBY incurs runtime and binary storage overheads of 2.8% and 0.6% for the RSA algorithm in cryptographic libraries, respectively. Overall, HoBBY provides strong security guarantees for secret-dependent branches within the TEE with acceptable overhead.

ACKNOWLEDGEMENT

We thank the anonymous reviewers for their valuable comments. This work was supported in part by the National Natural Science Foundation of China (Grant No. 62372258) and the Singapore Ministry of Education (Grant No. T2EP20222-0026). Chang Liu was supported by China Scholarship Council (Student No. 202406210249).

REFERENCES

- [1] A. Ahmad, B. Joe *et al.*, “OBFUSCURO: A Commodity Obfuscation Engine on Intel SGX,” in *Network and Distributed System Security Symposium (NDSS)*, 2019.
- [2] A. C. Aldaya, B. B. Brumley *et al.*, “Port Contention for Fun and Profit,” in *Symposium on Security and Privacy (SP)*, 2019, pp. 870–887.
- [3] AMD, “AMD64 Architecture Programmer’s Manual Volume 2: System Programming,” <https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24593.pdf>, 2024.
- [4] F. Brasser, U. Müller *et al.*, “Software Grand Exposure: SGX Cache Attacks Are Practical,” in *USENIX Workshop on Offensive Technologies (WOOT)*, 2017.
- [5] Y. Chen, L. Pei, and T. E. Carlson, “AfterImage: Leaking Control Flow Data and Tracking Load Operations via the Hardware Prefetcher,” in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2023, pp. 16–32.
- [6] S. Constable, J. Van Bulck *et al.*, “AEX-Notify: Thwarting Precise Single-Stepping Attacks through Interrupt Awareness for Intel SGX Enclaves,” in *USENIX Security Symposium (USENIX Security)*, 2023, pp. 4051–4068.
- [7] B. Coppens, I. Verbauwhede *et al.*, “Practical Mitigations for Timing-Based Side-Channel Attacks on Modern x86 Processors,” in *Symposium on Security and Privacy (SP)*, 2009, pp. 45–60.
- [8] S. Deng, M. Li *et al.*, “CipherH: Automated Detection of Ciphertext Side-channel Vulnerabilities in Cryptographic Implementations,” in *USENIX Security Symposium (USENIX Security)*, 2023, pp. 6843–6860.
- [9] S. Gast, J. Juffinger *et al.*, “SQUIP: Exploiting the Scheduler Queue Contention Side Channel,” in *Symposium on Security and Privacy (SP)*, 2023, pp. 2256–2272.
- [10] J. Götzfried, M. Eckert *et al.*, “Cache Attacks on Intel SGX,” in *Proceedings of the European Workshop on Systems Security (EuroSec)*, 2017, pp. 1–6.
- [11] B. Gras, K. Razavi *et al.*, “Translation Leak-aside Buffer: Defeating Cache Side-channel Protections with TLB Attacks,” in *USENIX Security Symposium (USENIX Security)*, 2018, pp. 955–972.
- [12] J. Gyselinck, J. Van Bulck *et al.*, “Off-Limits: Abusing Legacy x86 Memory Segmentation to Spy on Enclaved Execution,” in *Engineering Secure Software and Systems (ESSoS)*, 2018, pp. 44–60.
- [13] M. Hähnel, W. Cui, and M. Peinado, “High-Resolution Side Channels for Untrusted Operating Systems,” in *USENIX Annual Technical Conference (USENIX ATC)*, 2017, pp. 299–312.
- [14] A. Hajiabadi and T. E. Carlson, “Conjuring: Leaking Control Flow via Speculative Fetch Attacks,” in *Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [15] Intel, “Guidelines for Mitigating Timing Side Channels Against Cryptographic Implementations,” <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/secure-coding/mitigate-timing-side-channel-crypto-implementation.html>.
- [16] Intel, “Intel® 64 and IA-32 Architectures Software Developer Manuals,” <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, 2024.
- [17] S. Lee, M.-W. Shih *et al.*, “Inferring Fine-grained Control Flow Inside SGX Enclaves with Branch Shadowing,” in *USENIX Security Symposium (USENIX Security)*, 2017, pp. 557–574.
- [18] M. Lexa and G. Valle, “PRIMEX: rapid identification of oligonucleotide matches in whole genomes,” *Bioinformatics*, vol. 19, no. 18, pp. 2486–2488, 2003.
- [19] libjpeg, “libjpeg version 9f,” <https://www.ijg.org/files/jpegsrc.v9f.tar.gz>, 2024. [Online]. Available: <https://www.ijg.org/files/jpegsrc.v9f.tar.gz>
- [20] llvm, “Dfsan,” <https://clang.llvm.org/docs/DataFlowSanitizer.html>, 2020.
- [21] llvm, “llvm-project,” <https://github.com/llvm/llvm-project>, 2024.
- [22] Mbedtls, “Mbedtls version 2.6.1,” <https://github.com/Mbed-TLS/mbedtls/tree/mbedtls-2.6.1>, 2017. [Online]. Available: <https://github.com/Mbed-TLS/mbedtls/tree/mbedtls-2.6.1>
- [23] Mbedtls, “Mbedtls version 3.6.1,” <https://github.com/Mbed-TLS/mbedtls/tree/mbedtls-3.6.1>, 2024. [Online]. Available: <https://github.com/Mbed-TLS/mbedtls/tree/mbedtls-3.6.1>
- [24] A. Moghimi, G. Irazoqui, and T. Eisenbarth, “CacheZoom: How SGX Amplifies the Power of Cache Attacks,” in *International Conference on Cryptographic Hardware and Embedded Systems (CHES)*, 2017, pp. 69–90.
- [25] D. Moghimi, J. Van Bulck *et al.*, “CopyCat: Controlled Instruction-Level Attacks on Enclaves,” in *USENIX Security Symposium (USENIX Security)*, 2020, pp. 469–486.
- [26] I. Puddu, M. Schneider *et al.*, “Frontal Attack: Leaking Control-Flow in SGX via the CPU Frontend,” in *USENIX Security Symposium (USENIX Security)*, 2021, pp. 663–680.
- [27] M. Schwarz and D. Gruss, “How Trusted Execution Environments Fuel Research on Microarchitectural Attacks,” *IEEE Security & Privacy*, vol. 18, no. 5, pp. 18–27, 2020.
- [28] M. Schwarz, S. Weiser *et al.*, “Malware Guard Extension: Using SGX to Conceal Cache Attacks,” in *International Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2017, pp. 3–24.
- [29] J. Seo, B. Lee *et al.*, “SGX-Shield: Enabling Address Space Layout Randomization for SGX Programs,” in *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [30] D. Skarlatos, M. Yan *et al.*, “MicroScope: enabling microarchitectural replay attacks,” in *Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 318–331.
- [31] J. Van Bulck, F. Piessens, and R. Strackx, “SGX-Step: A Practical Attack Framework for Precise Enclave Execution Control,” in *Proceedings of the Workshop on System Software for Trusted Execution (SysTEX)*, 2017, pp. 1–6.
- [32] J. Van Bulck, F. Piessens, and R. Strackx, “Nemesis: Studying Microarchitectural Timing Leaks in Rudimentary CPU Interrupt Logic,” in *Proceedings of the SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 178–195.
- [33] J. Van Bulck, N. Weichbrodt *et al.*, “Telling Your Secrets without Page Faults: Stealthy Page Table-Based Attacks on Enclaved Execution,” in *USENIX Security Symposium (USENIX Security)*, 2017, pp. 1041–1056.
- [34] W. Wang, G. Chen *et al.*, “Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX,” in *Proceedings of the SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 2421–2434.
- [35] Y. Wang, R. Paccagnella *et al.*, “Hertzbleed: Turning Power Side-Channel Attacks Into Remote Timing Attacks on x86,” in *USENIX Security Symposium (USENIX Security)*, 2022, pp. 679–697.
- [36] J. Wichelmann, A. Rabich *et al.*, “Obelix: Mitigating Side-Channels Through Dynamic Obfuscation,” in *Symposium on Security and Privacy (SP)*, 2024, pp. 189–189.
- [37] L. Wilke, J. Wichelmann *et al.*, “SEV-Step: A Single-Stepping Framework for AMD-SEV,” *International Conference on Cryptographic Hardware and Embedded Systems (CHES)*, 2024.
- [38] H. Winderix, J. T. Mühlberg, and F. Piessens, “Compiler-Assisted Hardening of Embedded Software Against Interrupt Latency Side-Channel Attacks,” in *European Symposium on Security and Privacy (EuroS&P)*, 2021, pp. 667–682.
- [39] wolfSSL, “wolfssl version 5.7.2-stable,” <https://github.com/wolfSSL/wolfssl>, 2024. [Online]. Available: <https://github.com/wolfSSL/wolfssl>
- [40] Y. Xu, W. Cui, and M. Peinado, “Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems,” in *Symposium on Security and Privacy (SP)*. IEEE, 2015, pp. 640–656.
- [41] Y. Yarom, D. Genkin, and N. Heninger, “CacheBleed: a timing attack on OpenSSL constant-time RSA,” *Journal of Cryptographic Engineering*, vol. 7, pp. 99–112, 2017.
- [42] H. Yavarzadeh, A. Agarwal *et al.*, “Pathfinder: High-Resolution Control-Flow Attacks Exploiting the Conditional Branch Predictor,” in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 3, 2024, pp. 770–784.
- [43] J. Yu, T. Jaeger, and C. W. Fletcher, “All Your PC Are Belong to Us: Exploiting Non-control-Transfer Instruction BTB Updates for Dynamic PC Extraction,” in *Proceedings of the Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–14.
- [44] P. Zhang, C. Song *et al.*, “Klotski: Efficient Obfuscated Execution against Controlled-Channel Attacks,” in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 1263–1276.
- [45] X. Zhang, Z. Zhang *et al.*, “SegScope: Probing Fine-grained Interrupts via Architectural Footprints,” in *International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 424–438.